

Witbank Elite Car Wash

INT316D — Web Development Group Project

Semester 1, 2026

Individual Reflection Statements

Student Name: C.W MYA

Student ID: 223842763

Role: Backend Developer (Spring Boot)

Introduction

This semester has been a defining chapter in my journey as a software developer. The Witbank Elite Car Wash project was unlike any academic exercise I had encountered before — it demanded not just technical competency, but the discipline and architectural thinking of a professional engineer. As the Backend Developer for our team of ten, I was responsible for the heart of the application: the Spring Boot server, the RESTful endpoints, the service layer business logic, and the in-memory data management that powered every booking, notification, and report the system generated. This reflection documents the technical work I carried out, the challenges I overcame, the lessons the collaboration taught me, and the direction this experience points me toward in my career.

Technical Contribution and Implementation

My primary responsibility was building and structuring the Spring Boot 3.2.5 application that forms the backbone of the Witbank Elite system. The project brief explicitly prohibited the use of the JEE framework and Glassfish application server — a constraint that, far from being limiting, introduced me to a modern Java development paradigm I now consider vastly superior for greenfield projects. Spring Boot's auto-configuration model, embedded Tomcat server, and convention-over-configuration philosophy allowed me to stand up a production-ready web server in hours rather than days.

I designed and implemented the core domain models: Booking, ServicePackage, InventoryItem, and NotificationLog. Each model was crafted to be a clean Plain Old Java Object (POJO), keeping the data layer decoupled from business logic. The BookingService class became the most complex component I built — it encapsulates all business rules including booking creation, status transitions, customer visit counting, VIP tier logic (customers with three or more bookings are automatically flagged as VIP), revenue calculation, and the triggering of SMS and email notifications through the NotificationService.

Implementing the notification system was a technically rewarding challenge. I integrated JavaMailSender with an SMTP configuration pointing to a Gmail relay, enabling real HTMLformatted booking confirmation emails. For SMS, I simulated delivery via an internal

logging mechanism — a pragmatic decision given the cost of real SMS gateway APIs for an academic project — but I architecturally structured the NotificationService so that swapping in a real provider like Twilio requires only a one-class change. This design-for-replaceability principle, which I learned from studying clean architecture patterns, significantly influenced how I structured all service boundaries.

I also built the BookingController and AdminController, implementing the MVC pattern strictly. Controllers in my implementation are deliberately thin — they handle HTTP request mapping and delegate entirely to the service layer. This separation meant that when the Frontend Developer needed to change a form field, no business logic was touched, and when the Database Manager later wanted to wire up persistent storage, the service interfaces remained unchanged.

Collaboration and Group Dynamics

Backend development in a ten-person team creates unique integration challenges. My code was the runtime environment in which everyone else's work executed — frontend templates, database queries, notification triggers — which meant that my architectural decisions had cascading consequences for the whole team. I learned to communicate schema changes through shared interface documents rather than direct code changes, reducing integration conflicts on our shared GitHub repository.

The most productive collaboration was with the Database Manager. Although the initial implementation used in-memory storage (Java collections within the service layer), I designed the data access patterns so they could be extracted into a repository interface with minimal refactoring. This forward-thinking approach — building for the next iteration rather than only the current one — is a practice I observed in senior developers' codebases I studied while researching Spring best practices.

I also worked closely with the Tester, who identified several edge cases in the booking flow — such as duplicate submissions on form re-load and time slot validation — that I had not anticipated. Addressing these through server-side validation in the BookingController strengthened my appreciation for having a dedicated QA perspective in any development team.

Lessons Learned and Future Outlook

The most significant technical lesson of this semester is the value of the embedded server model. Seeing our application package as a single executable JAR that runs with `java -jar`, without any external application server configuration, was a revelation. It demystified the deployment gap between development and production that had always felt opaque to me when working with traditional JEE setups.

I also gained a deep appreciation for Spring's dependency injection container. By defining service beans and injecting them through constructor injection, I built a codebase where every component is independently testable and replaceable. This understanding transforms how I think about software architecture — not as a collection of classes, but as a graph of collaborating components with clearly defined contracts.

Looking ahead, I intend to explore Spring Data JPA and database persistence as the natural next step for this project. I also want to study Spring Security to implement authentication for the admin dashboard, which currently has no access control — a production gap I am aware of and plan to address.

Conclusion

Building the backend of Witbank Elite Car Wash gave me the most concentrated professional experience of my academic career. I now understand what it means to design a system that others depend on, to write code that is not just correct but maintainable, and to make architectural decisions with consequences that extend far beyond the current sprint. I am grateful for the team, the project brief, and the challenging constraints that pushed me beyond what I thought I could build.